PRINCIPLES OF COMPUTER PROGRAMMING (2440099)

Hello World, Toolchain, and How a Program Becomes a Program

Week 1

TODAY'S AGENDA

Four things by the end of today

1. **The four-stage pipeline**

   Preprocess -> compile -> assemble -> link.

2. **Translation units**

   Headers declare. .cpp files define.

3. **main, argc, argv — and warnings**

   Your first pointer, before the word is formal.

4. **Compiler warnings & build presets**

   Reading a real warning, debug vs strict.

FIRST LOOK

What a C++ program actually looks like

```
#include "grid.h"
#include <iostream>
int main() {
    Grid room(32, 12);          // a comment
    std::cout << room.to_string();
    return 0;
}
```

`#include`
Pulls in outside code.

`int main() { }`
Runs first, always.

`;`
Ends one statement.

`{ }`
Groups statements together.

`// text`
Ignored by the compiler.

`std::cout <<`
Prints text (week 3).

YOUR DEVELOPMENT ENVIRONMENT

VS Code, its extensions — and Cline

1. **Remote - WSL**
   Connects VS Code (Windows) to the Linux compiler.
2. **C/C++ + CMake Tools**
   Language support and build presets.
3. **GDB (via C/C++ extension)**
   Powers the F5 debugger from ten minutes ago.
4. **Cline**
   This course's AI assistant -- panel on the right.

Cline + DeepSeek, BYOK

Your own DeepSeek key. No shared account -- configured in Cline's settings, never a repo file.

Installed this week.
Not used until week 4.

AI-off through week 3, checked in-lab. Ready week 4 -- not before.

BEFORE LAB 0 — STEP 1 OF 4

Installing WSL2, with Debian

1

- Open PowerShell as Administrator, then restart when asked.
- First launch asks for a Linux username and password — not your Windows one.
- Confirm VERSION 2 before moving on.

```
$ wsl --install -d Debian

$ wsl -l -v
  NAME      STATE    VERSION
  Debian    Running  2
```

Preparing Debian for C++

2

- Update the package index first.
- One line installs everything this course needs.
- Verify the versions before you trust them.

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install -y build-essential \
      gdb cmake git valgrind clang-format

$ g++ --version      # want 11+
$ cmake --version    # want 3.20+
```

Clone the starter project

3

- This course hosts code on Gitea, not GitHub.
- **Clone into the Linux filesystem — never /mnt/c/...**
- **/mnt/c builds are 5–10x slower**, every time.

```
$ cd ~
$ mkdir -p 2440099 && cd 2440099
$ git clone \
    https://git.pdulab.net/pdu/2440099 \
    starter
```

BEFORE LAB 0 — STEP 4 OF 4

Open it in VS Code

4

- code . opens VS Code, connected to WSL.
- Bottom-left corner should read WSL: Debian.
- Say yes to the recommended extensions prompt.

```
$ cd starter
$ code .
```

You just typed:

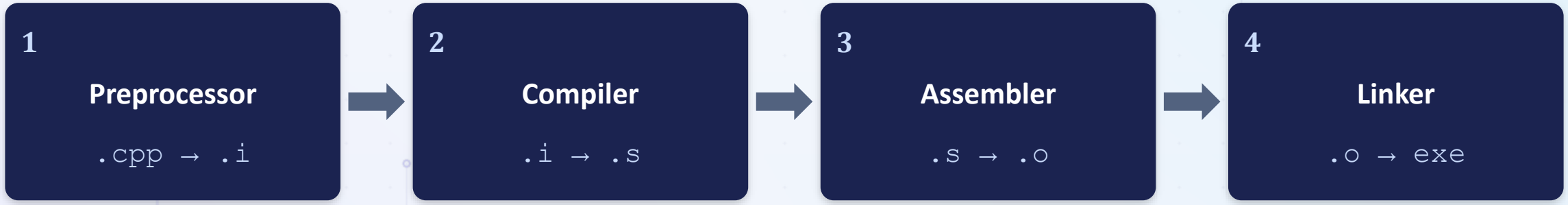
```
g++ hello.cpp -o hello
```

What actually happened in between?

Today, we open that box.

THE BIG PICTURE

Four independent stages, four different errors



Each stage is a separate program, with its own failure mode.

STAGE 1 OF 4

The preprocessor: pure text substitution

1

- Runs before the compiler even looks at your code — it has no idea what C++ is.
- #include pastes a file's contents in, verbatim, at that exact line.
- #define does plain text substitution.
- Output: an expanded source file (.i) — often thousands of lines from a handful of #include lines.

```
$ g++ -E src/main.cpp -Iinclude -Isrc \
      -o /tmp/main.i
$ wc -l /tmp/main.i
3482 /tmp/main.i

# almost all of it came from <iostream>
# and the two course headers.
```

STAGES 2 & 3 OF 4

The compiler, then the assembler

2 The compiler

- .i → .s (assembly): human-readable-ish CPU instructions.
- Syntax and type checking happen HERE — a missing semicolon or a type mismatch is caught at this stage.
- You will never hand-write assembly in this course.

3 The assembler

- .s → .o: real machine code, but not yet a program.
- Can contain calls to functions defined in OTHER files — those are left as blanks.
- Each .cpp is assembled completely independently.

STAGE 4 OF 4

The linker: where “undefined reference” comes from

4

- Combines every .o file the program needs into one executable.
- Resolves every blank the assembler left — a call gets patched to the real address of its definition.
- If nothing, anywhere, defines a function you called: linker error, not a compiler error.

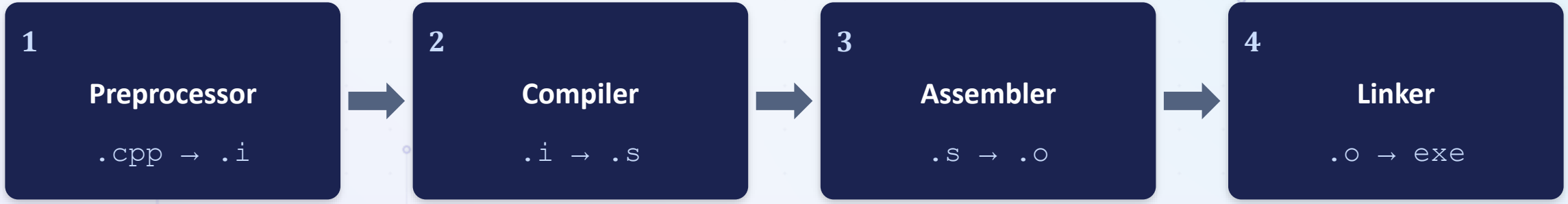
```
int mystery();
int main() { return mystery(); }

$ g++ tmp_bad.cpp -o bad
undefined reference to `mystery()'
```

Every stage before this one succeeded -- nothing anywhere defines the function you called.

RECAP

One .cpp, one program — by four stages



Next: what a .cpp file actually IS, once every #include has been expanded.

A PRECISE DEFINITION

Translation units

One .cpp file, plus everything it pulls in via #include, after the preprocessor has finished.

- The compiler compiles ONE translation unit at a time.
- It has no idea any other .cpp file in the project exists.
- So how does main.cpp know Grid's constructor exists, if grid.cpp is compiled completely separately?

Answer: the header is a promise. The linker keeps it.